

Verified Agent Transport (VAT)

Technical Specification — v1.0 — October 9, 2026

*AI Agent Wrapper Service on the QueryBook + TCP/IP Hybrid Transport
(UFCS-FQL/1)*

Status: design specification, v1.0 (October 9, 2026). Not yet implemented.

Audience: engineers (and Claude Code) building the service. **Builds on:**

- QueryBook prototype v9.64 (querybook/QueryBook-v9.64): Fact Units, UFCS store, FQL, Agent Gateway, security log, certificates, MCP server.
- The UFCS-FQL/1 frame format in `ufcs-transport-lab/src/Protocol.php` and `Frame.php`.

1. Purpose

VAT is a transport service for AI agents. Every message an agent sends or receives travels as UFCS-FQL frames through a QueryBook **sidecar** placed next to the agent. The sidecar:

1. **Verifies** every factual claim against the UFCS store (verified / contradicted / unknown).
2. **Binds actions to intent.** The agent declares a plan; each tool call is checked against the plan and the agent's allow-list; the reported result is compared with the tool's own record.
3. **Records** every frame and verdict in a tamper-evident hash chain with Merkle checkpoints.
4. **Moves data by reference.** Content-addressed Fact Units are exchanged with a have/want protocol, so data the receiver already holds is never re-sent.
5. **Certifies** each session with an Agent Session Certificate built over the Merkle root of the session.

The sidecar certifies the **record**, not the agent's mind. A certificate states what was checked and what the checks found. It never states that an agent "is honest".

Non-goals

- Modifying agent models or frameworks. Agents keep speaking MCP, A2A or HTTP; the sidecar translates.
- Replacing TLS or QUIC. VAT frames ride inside encrypted connections.
- Asserting any bandwidth or latency improvement without measurement (see §13).

2. Architecture

```

Agent A --MCP/A2A/HTTP--> [VAT Sidecar A]
                        || VAT frames over QUIC (or TLS/TCP)
                        \=====> [VAT Sidecar B] --> Agent B / tool
Sidecar A: verify, bind, sign, log, certify      Sidecar B: verify, log
Both sidecars --> UFCS Verification Service
Shared: Ledger + Certificates; Control Plane (policy)
(see drawings, FIG. 1)

```

Component	Role	Reuses
Sidecar (qb_vat_sidecar.py)	Co-located with one agent. Adapters (MCP, A2A, HTTP), frame codec, claim extraction, verifier client, intent binding, ledger writer, certificate issuer.	qb_mcp, qb_shield, qb_seclog, qb_openclaw
Egress lock	OS or cluster network policy: the agent can reach only its sidecar.	nftables / Kubernetes NetworkPolicy
UFCS Verification Service (qb_vat_verify.py)	Shared, read-only fact checking: fingerprint lookup and FQL contradiction query.	ufcs_store, FQL
Ledger	Hash chain plus write-once Merkle checkpoints per sidecar; optional mirror and anchor.	qb_seclog, qb_mirror, qb_nft
Control plane (qb_vat_control.py)	Distributes allow-lists, honeytokens, verification tiers and verifier keys. Pull-only, signed bundles.	qb_shield policy format

3. Wire format

VAT uses **UFCS-FQL/1** unchanged (Protocol::VERSION = 1, magic 0xF051). A frame is:

```

+-----+-----+-----+-----+
| Header (32 bytes) | Meta (MetaLen) | Payload          | Footer          |
|                   | UTF-8 JSON      | (PayloadLen)     | CRC32 (4) [+ Ed25519 64] |
+-----+-----+-----+-----+

```

Header layout, big-endian (pack('nCCCCnNNJNN')):

Field	Bytes	Notes
magic	2	0xF051 (resynchronization anchor)
version	1	1

Field	Bytes	Notes
msgType	1	See §3.1. VAT uses 16–31; 0–2 keep their existing meanings.
contentType	1	TEXT / IMAGE / VIDEO / AUDIO / MIXED
compression	1	Existing codes. 9 = UFCS_DICT for record batches.
flags	2	Priority bits, SIGNED, END_OF_STREAM, ACK_REQUESTED; VAT adds 0x0040 VERIFIED_HOP
payloadLen	4	
metaLen	4	
msgId	8	Unique per sender session
sequence	4	Per-lane sequence number
reserved	4	VAT: low 16 bits = session number, high 16 = hop count

CRC32 (IEEE) covers header + meta + payload. When SIGNED is set, an Ed25519 signature over header + meta + payload + CRC follows.

3.1 VAT message types

Code	Name	Lane	Payload	Required meta
16	CLAIM	normal	UFCS packet(s) (JSON or compact encoding)	agent, session, claim_ids[], assertion_type (fact / opinion / plan / prediction)
17	QUERY	normal	FQL text	agent, session, min_trust
18	ANSWER	normal	Results + citations	response_provenance_hash
19	INTENT	control	Declared plan (ordered steps, each naming a tool and parameter constraints)	intent_id
20	ACTION	control	Tool call	intent_id, step
21	RESULT	control	The tool's own result record + the agent's reported result	intent_id, step, tool_record_hash
22	VERDICT	control	Verdict records (§5.3)	verifier, signed=true
23	DATA	bulk	Fact Unit batch or media chunk	fingerprints[] or stream_id
24	CERT	control	Agent Session Certificate (§7)	session
25	HAVE	control	List of fingerprints offered	batch_id

Code	Name	Lane	Payload	Required meta
26	WANT	control	Subset of fingerprints requested	batch_id
27	TEXT	normal	Free agent message text (claims extracted by the sidecar)	agent, session

Unknown msgType values must be dropped and logged, never forwarded.

3.2 Lanes

Lane	Carries	QUIC mapping	TLS/TCP fallback
control	INTENT, ACTION, RESULT, VERDICT, CERT, HAVE, WANT	Bidirectional stream 0 (highest priority)	Connection 1
normal	CLAIM, QUERY, ANSWER, TEXT	Bidirectional stream 4	Connection 2
bulk	DATA	Unidirectional streams, one per transfer, lowest priority	Connection 3

A verdict must never wait behind bulk data. Under QUIC this follows from independent streams; under TCP, from separate connections, as in the lab's Transport class.

4. Transport binding

- **Phase 1 (required):** TLS 1.3 over TCP using Python's standard `ssl` module, three connections per peer as in `ufcs-transport-lab/src/Transport.php`. Mutual TLS between sidecars; certificates issued by the control plane.
- **Phase 2:** QUIC / HTTP/3 using `aioquic`, as an optional dependency (QueryBook is otherwise standard-library only). Lanes map to streams per §3.2. Fall back to Phase 1 automatically when UDP/443 is blocked.
- **Controlled networks only:** the IP-level semantic header of the TCP/IP hybrid protocol may be used where every middlebox is operator-controlled. Never rely on it across the public internet.

5. Claim verification

5.1 Claim extraction

The sidecar converts outbound TEXT into CLAIM frames:

1. Split the text into sentences. Classify each as fact, opinion, plan or prediction with deterministic rules: modal verbs, first-person intent, future tense, evaluative adjectives.
2. For fact sentences, run QueryBook query understanding (qb_query) to produce subject / predicate / object / polarity triples, mapped to canonical identifiers.
3. Build a UFCS packet per triple and compute its semantic fingerprint: SHA-256 over the normalized subject, predicate, object and polarity.
4. Record **coverage** = fact-sentence characters successfully converted ÷ total fact-sentence characters.

Sentences that cannot be converted are marked unchecked, never verified.

5.2 Verification algorithm (Verification Service)

```
verify(claim):
    if store.has_fingerprint(claim.fp) and record.trust >= tier.min_trust:
        return VERIFIED, cite=record.fuid
    rivals = FQL: MATCH subject=claim.s, predicate=claim.p
                WHERE certification.trust_score >= tier.min_trust
                EXCLUDE object=claim.o
    if rivals and max(rival.trust) > claim_source_trust:
        return CONTRADICTED, cite=argmax_trust(rivals).fuid
    return UNKNOWN
```

- **Opposite polarity** with the same subject, predicate and object is CONTRADICTED.
- opinion, plan and prediction claims are never verified or contradicted; they are labelled.
- **Verification tiers** (from the control plane): — full: every claim checked. — sampled: a deterministic sample keyed on hash(session, claim_id) mod N. — fingerprint_only: privacy mode, §5.4.

5.3 Verdict record (VERDICT payload, JSON)

```
{
  "claim_id": "c-000123",
  "fingerprint": "9beedc0f...",
  "verdict": "verified | contradicted | unknown | unchecked | labelled_opinion",
  "cite": "fuid-d8506d27...",
  "trust": 0.776,
  "tier": "full",
  "verifier": "vat-verify-01",
  "ts": "2026-10-09T18:22:04Z",
  "sig": "ed25519:..."
}
```

The verifier signs every verdict with Ed25519. The sidecar sets VERIFIED_H0P on the forwarded frame and attaches the verdict IDs in meta.

5.4 Privacy mode (fingerprint_only)

- The sidecar sends only fingerprints to the Verification Service. The service answers verified or unknown from fingerprint lookup alone.
- A contradiction check needs only (subject, predicate), sent as their canonical identifiers. Claim text and objects never leave the sidecar.

5.5 Policy on delivery

Verdict	Default action (configurable per agent)
verified	Deliver with citation
unknown	Deliver, labelled "unverified"
contradicted	Block, or deliver with the contradicting citation attached (annotate mode)
unchecked	Deliver, counted against coverage

6. Intent binding

1. The agent, or the sidecar's adapter on its behalf, sends INTENT before acting: an ordered list of steps {tool, param_constraints}.
2. Each ACTION is checked against: — the agent's allow-list (Agent Gateway, qb_shield); — the open intent: the tool must be in the plan, and parameters must satisfy the constraints. An action outside the intent gives out_of_plan; the policy is either block or allow-and-flag.
3. The tool's own result record is hashed (tool_record_hash). When the agent later reports the outcome, the sidecar compares the report with the record. A mismatch gives report_mismatch, which is logged and counted in the certificate.
4. Honeycode, honeytokens and tripwires (qb_shield) apply unchanged. A trap hit isolates the agent to the decoy.

7. Agent Session Certificate (CERT payload)

```
{
  "type": "agent_session_certificate",
  "version": 1,
  "agent": "openclaw@host-17",
  "session": 4211,
  "opened": "2026-10-09T18:00:00Z",
  "closed": "2026-10-09T18:42:10Z",
  "counts": {
    "claims": 212, "verified": 151, "contradicted": 3, "unknown": 41, "unchecked": 17,
    "opinions_labelled": 22, "actions": 38, "in_plan": 37, "out_of_plan": 1, "report_mismatch": 0
  },
  "coverage": 0.92,
  "tier": "full",
}
```

```

"merkle_root": "b7c1...",
"checkpoint_ids": ["cp-000311", "cp-000312"],
"verifier_keys": ["ed25519:..."],
"statement": "This certificate records what was checked in this session and what the checks found. It does not
assert that the agent is honest or that its content is true.",
"sig": "ed25519:..."
}

```

- The certificate is sealed as a Fact Unit and written to the ledger.
- Optionally it is certified on-chain with qb_nft: one token per certificate hash, test networks unless elected.
- **Independent verification:** recompute the Merkle root from the session's ledger entries and check it against the certificate and any anchor.

7.1 Agent reputation

Each agent has an evidence pair (α, β) , using the Fact Unit confidence model:

- $\alpha +=$ verified claims + in-plan actions
- $\beta +=$ contradicted claims + out_of_plan + report_mismatch (each weight configurable)
- Unknown and unchecked claims do not change the pair.

Score = $\alpha / (\alpha + \beta)$, with a credible interval. Below the policy threshold, the control plane can lower the agent's tier, require approval, or restrict its allow-list. Restriction always requires a recorded operator decision, consistent with Default-on-Deny.

8. Data transport: have/want exchange

Sender		Receiver
HAVE {batch_id, fingerprints[n]}	→	
		← WANT {batch_id, fingerprints[k ≤ n]} (those it lacks)
DATA {fingerprints[k]} (bulk)	→	
		← ACK {batch_id, stored[k]}

- Fingerprint lists longer than 4,096 entries are sent as a Bloom filter with a false-positive rate $\leq 1e-6$, followed by an exact list for the hits.
- **Query-driven fetch:** a receiver can send QUERY with FQL (e.g. MATCH subject=X WHERE trust $\geq$ 0.7). The answer returns fingerprints, followed by HAVE/WANT.
- Every received record is verified (signature, provenance chain, fingerprint recomputed from content) before it enters the receiver's store.

9. Sidecar and egress enforcement

Linux host (nftables): the agent runs as user agent; only the sidecar port is reachable.

```
table inet vat {
  chain output {
    type filter hook output priority 0;
    meta skuid "agent" ip daddr 127.0.0.1 tcp dport 8790 accept
    meta skuid "agent" drop
  }
}
```

Kubernetes: the agent and sidecar share a pod. A NetworkPolicy denies all egress from the agent container except localhost:8790. Only the sidecar has an egress rule to peer sidecars and the Verification Service.

Windows / desktop: Windows Firewall outbound block for the agent executable, plus an allow rule for 127.0.0.1:8790.

Adapters:

- **MCP:** the sidecar is the agent's MCP server and forwards tool calls as ACTION frames.
- **A2A:** messages are wrapped as TEXT/CLAIM frames.
- **HTTP:** a forward proxy on 127.0.0.1:8790.

10. Configuration (vat.json)

```
{
  "agent": "openclaw@host-17",
  "listen": "127.0.0.1:8790",
  "peers": ["vat://sidecar-b.example:8791"],
  "transport": {"prefer": "quic", "fallback": "tls-tcp", "lanes": 3},
  "verify": {"service": "https://vat-verify.internal:8443", "tier": "full", "min_trust": 0.5},
  "policy": {"contradicted": "annotate", "out_of_plan": "flag", "report_mismatch": "flag"},
  "ledger": {"dir": "~/.querybook/vat-ledger", "checkpoint_every": 256, "mirror": null, "anchor": false},
  "certify": {"on_close": true, "nft": false},
  "privacy": {"fingerprint_only": false}
}
```

11. APIs

Sidecar, local:

- POST /vat/text
- POST /vat/intent
- POST /vat/action
- GET /vat/session/{id}/certificate
- GET /vat/status

Verification Service:

- POST /verify — batch of claims or fingerprints in, signed verdicts out
- GET /verify/key

Control plane:

- GET /policy/{agent} — signed bundle
- GET /keys

Verifier CLI:

- `qb_vat verify-cert <cert.json> --ledger <dir>` — recomputes the Merkle root; checks signatures and any anchor.

12. Implementation plan (for Claude Code)

Step	Deliverable	Acceptance test
1	<code>qb_vat_frames.py</code> : Python codec for UFCS-FQL/1 plus VAT types 16-27	Byte-identical round-trip with frames produced by <code>ufcs-transport-lab</code> (PHP)
2	<code>qb_vat_verify.py</code> : verification service over <code>ufcs_store</code>	Known fact gives verified; same subject/predicate with a different object gives contradicted with a citation; novel fact gives unknown; opinions labelled
3	<code>qb_vat_sidecar.py</code> : TLS/TCP, three lanes, MCP adapter, claim extraction, ledger via <code>qb_seclog</code>	An OpenClaw agent runs end to end through the sidecar; every frame appears in the ledger; the chain verifies
4	Intent binding	An out-of-plan tool call is flagged; a falsified result report gives <code>report_mismatch</code>
5	Have/want	Re-sending an already-held batch transfers zero DATA frames
6	Certificate + <code>verify-cert</code> CLI	Independent recomputation of the Merkle root matches; tampering with one ledger entry is detected
7	Egress lock recipes	With the lock active, the agent cannot reach any host except its sidecar (test with <code>curl</code>)
8	QUIC binding (optional)	Lanes over QUIC streams; automatic TLS/TCP fallback when UDP is blocked
9	Self-tests	<code>qb_selftest.run_all()</code> includes VAT modules; all pass

Code rules:

- Standard library only, except optional aioquic.
- Deterministic: same inputs give the same verdicts and certificates.
- Each module has a `qc()` self-test, following the existing prototype pattern.

13. Security considerations and limits

- **Claim extraction is imperfect.** Coverage is always reported; unchecked content is never presented as verified.
- **Unknown $\neq$ false.** Agents are not penalized for novel but unverified statements.
- **Evasion by vagueness.** Track and report the hedging rate (opinion- and plan-labelled sentences as a share of all sentences).
- **Verifier trust.** Verdicts are only as good as the UFCS store and its trust ceilings. The certificate lists verifier keys and tier.
- **Side channels.** Only traffic through the sidecar is covered. Deployment must enforce the egress lock (§9).
- **Latency.** Full verification adds round-trips. Use tiers and per-agent policy; measure before stating any figure.
- **Bandwidth claims.** Savings from have/want depend on how much data repeats. They must be measured per workload under the trial architecture before any figure is published.
- **Observation only.** VAT records, contains and labels. It takes no action against external systems.